

Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики
Центр обучения проектированию и разработке игр

Выпускная квалификационная работа бакалавра

Предобработка мешей для использования в ускоряющих структурах

Автор:

Студент 251 группы
Мусько Денис Владимирович

Научный руководитель:

Санду Роман Александрович



Москва 2026

Аннотация

Предобработка мешей для использования в ускоряющих структурах *Мусько Денис Владимирович*

Работа посвящена ускорению трассировки лучей в реальном времени за счёт предобработки ассетов перед их подачей в структуру нижнего уровня (BLAS). Основная гипотеза состоит в том, что графический конвейер часто выделяет отдельный меш-ассет (и отдельный BLAS) под каждый уникальный материал (например, дерево или металл). Если элементы такого материала разбросаны по всему объёму крупной модели, то ограничивающий объём (AABB) соответствующего BLAS будет сопоставим с размерами всей модели. В результате в структуре верхнего уровня (TLAS) возникает множество огромных и полностью перекрывающихся инстансов — по одному на каждый материал. При трассировке луча это приводит к необходимости пересекать почти все BLAS сцены, что может существенно снижать производительность.

Предложен и реализован метод глобальной пространственной кластеризации ассетов. Треугольники всех материалов крупной модели извлекаются, объединяются в единый массив и сортируются по 63-битному Morton-коду центроида. Затем массив разделяется на K пространственно-компактных кластеров (фактически, новых мини-ассетов). Каждый такой кластер регистрируется в TLAS как самостоятельный BLAS с маленьким и плотно прилегающим AABB. Метод применяется во время работы приложения (на этапе инициализации сцены), не требует вмешательства в драйвер графического вендора и гарантирует попиксельную идентичность финального изображения.

Метод реализован в семпле `testGI` движка DagorEngine на модели `House, Ko Bulon Don Urak Lawoi Village, Thailand` ($\sim 1,2 \cdot 10^6$ треугольников, 16 инстансов). На целевой сцене получено восьмикратное снижение медианного времени трассировки теней `rtsm::do_trace`: с 8 мс до 1 мс на кадр. Корректность метода подтверждена визуально через сравнение Acceleration-Structure-картин в Nsight Graphics: вместо «шумовой» картины перекрывающихся огромных BLAS-инстансов наблюдаются мелкие пространственно-связные цветные области, соответствующие локализованным неперекрывающимся BLAS-кластерам.

Abstract

This work focuses on accelerating real-time ray tracing by preprocessing mesh geometry before it is submitted to the Bottom-Level Acceleration Structure (BLAS). The central hypothesis is that the graphics pipeline often allocates a separate mesh asset (and thus a separate BLAS) for each unique material (e.g., wood or metal). If the elements of such a material are scattered across a large model, the bounding volume (AABB) of its corresponding BLAS will span the entire model. As a result, the Top-Level Acceleration Structure (TLAS) is populated with multiple enormous and completely overlapping leaf nodes — one for each material. During ray traversal, this forces the intersection of almost all BLASes in the scene, critically degrading performance.

A method for global spatial geometry clustering is proposed and implemented. The triangles from all materials of a large model are extracted, combined into a single array, and sorted using a 63-bit centroid Morton code. The array is then partitioned into K spatially compact clusters (effectively new mini-assets). Each cluster is registered in the TLAS as an independent BLAS with a small, tightly fitting AABB. The method is

applied at runtime during scene initialization, requires no modifications to the vendor driver, and guarantees pixel-perfect visual equivalence of the rendered image.

The method is implemented in the `testGI` sample of the `DagorEngine` on the `House, Ko Bulon Don Urak Lawoi Village, Thailand` model ($\sim 1.2 \cdot 10^6$ triangles, 16 instances). On the target scene the method delivers an eight-fold reduction of the median ray-traced shadow trace time `rtsm::do_trace`: from 8ms down to 1ms per frame. The validity of the method is confirmed visually by comparing the Acceleration-Structure views in `Nsight Graphics`: instead of a noisy mixture of overlapping huge BLAS instances, small spatially-connected coloured regions corresponding to localised, non-overlapping BLAS clusters become visible.

Содержание

1	Введение	6
1.1	Актуальность	6
1.2	Объект и предмет исследования	7
1.3	Ключевые понятия	7
1.4	Постановочная гипотеза	7
1.5	Цель и задачи	8
1.6	Научная новизна	8
1.7	Практическая значимость	8
2	Постановка задачи	9
2.1	Формальная модель входа	9
2.2	Модель построения BLAS	9
2.3	Гипотеза проблемы	9
2.4	Содержательная задача	10
2.5	Технические требования	11
2.6	Критерии приёмки	11
3	Обзор существующих решений	12
3.1	Классические top-down SAH-билдеры	12
3.2	HLBVH и LBVH: параллельная сборка по Morton-кривой	12
3.3	SBVH и SAH-guided pre-splitting	13
3.4	Метрики качества BVH	13
3.5	Альтернативные ускоряющие структуры	13
3.6	Промышленные руководства и анализаторы	14
3.7	Сводное сравнение	14
3.8	Выводы по обзору	14
4	Исследование и построение решения задачи	16
4.1	Декомпозиция задачи	16
4.2	P1. Pipeline формирования BLAS в DagorEngine	16
4.3	P2. Алгоритм глобальной пространственной кластеризации	17
4.3.1	Идея	17
4.3.2	Шаги алгоритма	18
4.3.3	Сложность	19
4.3.4	Корректность	19
4.4	P3. Способ валидации	19
4.5	Сводная схема предлагаемого решения	20
4.6	Параметры алгоритма и выбор значений	20
5	Описание практической части	21
5.1	Архитектура реализации	21
5.2	Реализация алгоритмического модуля	21
5.2.1	Ключевые структуры данных	21
5.2.2	Основные функции	22
5.2.3	Аппаратные требования к буферам	22
5.2.4	Согласование K с лимитом 16-битного индекса	22
5.3	Конфигурация семпла	23
5.4	Тестовый стенд	23
5.4.1	Сцена	23

5.4.2	Тестовая модель House, Ko Bulon Don Urak Lawoi Village, Thailand	23
5.4.3	Условия замера	23
5.5	Результаты	24
5.5.1	Главный замер: время трассировки теней	24
5.5.2	Качественная картина Acceleration Structure	24
5.6	Обсуждение	24
5.6.1	Интерпретация ускорения	24
5.6.2	Соответствие критериям приёмки	24
5.6.3	Цена ускорения	25
5.6.4	Ограничения текущей реализации	26
6	Заключение	28
6.1	Решённая задача	28
6.2	Полученные результаты	28
6.3	Соответствие критериям приёмки	29
6.4	Научная новизна и практическая значимость	29
6.5	Направления дальнейшего развития	29
6.6	Итог	30

1 Введение

1.1 Актуальность

Трассировка лучей (ray tracing) в последнее десятилетие стала основным алгоритмом физически корректного рендеринга и активно проникает в реальное время благодаря появлению специализированных аппаратных блоков NVIDIA RT Cores и AMD Ray Accelerators, а также программных интерфейсов DirectX Raytracing (DXR) и Vulkan KHR_ray_tracing. Однако стоимость одного ray-primitive теста растёт линейно с числом примитивов сцены, поэтому без ускоряющей пространственной структуры данный подход непригоден для интерактивных применений [1].

Стандартом де-факто для таких структур стала иерархия охватывающих объёмов (Bounding Volume Hierarchy, BVH). При трассировке луч обходит дерево BVH и отбрасывает поддеревья, AABV которых не пересекается с лучом, что переводит сложность поиска ближайшего пересечения в среднем к $O(\log n)$. Качество BVH принято измерять поверхностно-площадной эвристикой (Surface Area Heuristic, SAH) [1, 2]:

$$c(N) = c_T \sum_{N_i \in \text{Internal}} \frac{SA(N_i)}{SA(N)} + c_I \sum_{N_l \in \text{Leaf}} \frac{SA(N_l)}{SA(N)} |N_l| \quad (1)$$

где:

- N — корневой узел поддерева;
- Internal и Leaf — множества внутренних (развилки дерева, хранящие AABV потомков) и листовых узлов (хранящие сами треугольники);
- $SA(X)$ — площадь поверхности AABV узла X ;
- c_T и c_I — константные стоимости обхода узла и теста пересечения с треугольником;
- $|N_l|$ — число треугольников в листе N_l .

Формула считает ожидаемую стоимость трассировки луча на основе геометрической вероятности: отношение площадей $SA(X)/SA(N)$ — это шанс того, что луч, прошедший через N , пересечёт и вложенный узел X . Если AABV дочерних узлов сильно перекрываются, их суммарная площадь увеличивается. Лучу чаще приходится обходить обе ветви, что делает рендеринг дороже.

В современных графических приложениях BVH строится в два уровня:

- **BLAS (Bottom-Level Acceleration Structure)** — структуры нижнего уровня. Каждая из них содержит ассет (вершины и индексы треугольников) отдельного меша или 3D-модели в локальной системе координат.
- **TLAS (Top-Level Acceleration Structure)** — структура верхнего уровня. Она не содержит ассетов напрямую, а состоит из инстансов (экземпляров) BLAS, составленных по сцене с определёнными матрицами трансформаций (позиция, вращение, масштаб).

Такое разделение позволяет быстро обновлять сцену при движении объектов (достаточно перестроить только TLAS), не перестраивая тяжелые BLAS для самих ассетов. Построение BLAS, как правило, выполняется драйвером графического вендора и недоступно для прямого вмешательства разработчиком: API DXR/Vulkan RT принимает на вход набор ассетов (vertex buffer + index buffer + диапазон), а внутреннюю стратегию построения выбирает драйвер. Это означает, что основной канал влияния на качество BLAS со стороны разработчика — **предобработка меша до его передачи билдеру**.

1.2 Объект и предмет исследования

Объект исследования – процесс построения и обхода ускоряющих структур данных (BVH) для аппаратной трассировки лучей в реальном времени.

Предмет исследования – методы и алгоритмы пространственной кластеризации и предобработки ассетов трёхмерных моделей, направленные на повышение качества структур нижнего уровня (BLAS) и ускорение рендеринга.

1.3 Ключевые понятия

BVH Bounding Volume Hierarchy. Двоичное дерево, в листьях которого хранятся примитивы сцены, а во внутренних узлах – AABB всех потомков.

AABB

Axis-Aligned Bounding Box. Параллелепипед со сторонами, параллельными осям координат, содержащий поддереву примитивов.

BLAS / TLAS

Bottom- и Top-Level Acceleration Structure – два уровня иерархии BVH в современных графических API (DXR / Vulkan RT). BLAS хранит сами ассеты модели, а TLAS организует их экземпляры на сцене.

RElem

Атомарная меш-структура исследуемого движка DagonEngine (`ShaderMesh::RElem`). Содержит ссылку на материал и непрерывный поддиапазон вершин/индексов общего буфера; по одному RElem соответствует одна `RaytraceGeometryDescription` на входе билдера.

Morton-код

Числовая характеристика 3D-точки, полученная чередованием битов её нормированных координат. Сортировка точек по Morton-коду эквивалентна обходу пространства по заполняющей кривой Z-order, что обеспечивает локальность соседних элементов в массиве.

SAH cost

Поверхностно-площадная стоимость BVH; общепринятая офлайн-мера качества дерева, коррелирующая с реальным временем трассировки [1].

RTSM

Ray Traced Shadow Map. Подсистема трассировки теней в DagonEngine, выполняющая ray-cast от каждого пикселя экрана в сторону источника света; целевая нагрузка для измерения ускорения.

1.4 Постановочная гипотеза

Графический конвейер часто выделяет отдельный меш-ассет (и, соответственно, отдельный BLAS) под каждый уникальный материал (например, дерево или металл). Если элементы такого материала разбросаны по всему объёму крупной модели, то ограничивающий объём (AABB) соответствующего BLAS будет сопоставим с размерами всей модели. В результате в структуре верхнего уровня (TLAS) возникает множество огромных и полностью перекрывающихся инстансов — по одному на каждый материал. При сборке TLAS его листовые узлы (инстансы BLAS) получаются огромными и

сильно перекрывающимися друг с другом. При трассировке луча алгоритм обхода вынужден проверять пересечение со всеми этими крупными BLAS, что может существенно снижать производительность рендеринга.

Гипотеза работы: *глобальная пространственная кластеризация ассетов крупной модели (объединение треугольников всех её материалов) с последующим разделением её на множество независимых, пространственно-компактных мини-ассетов (каждый из которых станет отдельным BLAS с плотно прилегающим AABB), выполненная на стороне CPU до подачи данных в билдер, демонстрирует устойчивое снижение времени аппаратной GPU-трассировки лучей на сценах с таким типом ассетов.*

1.5 Цель и задачи

Цель – разработать и экспериментально оценить алгоритм рантайм-предобработки меша, повышающий качество BLAS, построенного вендорским билдером, и достичь измеримого ускорения трассировки лучей в реальной сцене игрового движка без визуальных потерь.

Задачи:

1. Провести обзор существующих подходов к улучшению качества BVH (пространственная сортировка, SAH-управляемое пред-разрезание, кластеризация), оценить их применимость в условиях фиксированного DXR-pipeline.
2. Изучить цепочку формирования BLAS в DagorEngine и определить точку, в которой возможна предобработка меша без модификации драйвера.
3. Реализовать алгоритм глобальной пространственной кластеризации ассетов на стороне runtime: объединение всех треугольников модели в единый массив, сортировка по Morton-коду, разбиение на K пространственно-компактных кластеров с собственными `vertex / index` буферами.
4. Провести замер ускорения трассировки на сцене с «плохими» ассетами в семпле `testGI` и подтвердить визуальную корректность результата.
5. Сформулировать ограничения метода и направления дальнейшей работы.

1.6 Научная новизна

В отличие от классических подходов HLBVH [3] и SBVH-presplitting [2], работающих *внутри* билдера BVH, предлагаемый метод оперирует исключительно *до* вызова вендорского BLAS-билдера, выполняется на стороне runtime и явно учитывает специфический интерфейс DXR. Это позволяет получать ускорение, не имея исходного кода драйвера и не модифицируя ассет-пайплайн движка.

1.7 Практическая значимость

Метод применим как опционально включаемый этап подготовки динамических ассетов при инициализации BVH-контекста и не требует переекспорта ассетов. Он встраивается в существующий рантайм-код движка без изменения формата хранения мешей, что критично для проектов индустриального масштаба, использующих большие массивы готовых ассетов.

2 Постановка задачи

2.1 Формальная модель входа

Рассматривается **статическая трёхмерная сцена (или крупная модель)** S , готовая к загрузке в ускоряющие структуры для аппаратной трассировки (TLAS и BLAS).

На уровне данных графического движка S представляет собой набор из R независимых геометрий (меш-ассетов или **RElem** в терминологии DagorEngine [4]):

$$S = (V, \{M_k\}_{k=1}^R), \quad (2)$$

где

- $V \in \mathbb{R}^{n_v \times 3}$ – единый массив (буфер), содержащий пространственные 3D-координаты n_v уникальных вершин. Этот массив может быть общим (разделяемым) для всей сцены/ассета;
- $M_k \subset \{1, \dots, n_v\}^3$ – набор треугольников k -того меш-ассета (геометрии/материальной подгруппы). Каждый треугольник задаётся тройкой индексов, ссылающихся на координаты вершин в общем буфере V ;
- R – общее число изначальных логических кусков геометрии (например, разделённых по материалам).

В пайплайнах игровых движков графические дизайнеры зачастую разделяют сложные объекты не по пространственному признаку, а по логическому или материальному. Например, могут выделить в отдельный меш-ассет M_k все деревянные конструкции сложного здания или уровня. Геометрия может существовать в нескольких уровнях детализации (LOD), при этом для формирования структуры трассировки используется геометрия наивысшего качества.

2.2 Модель построения BLAS

При подготовке к аппаратному рейтрейсингу графический движок подаёт каждую геометрию M_k в драйвер как независимую сущность. В соответствии с этим билдер драйвера воспринимает каждый M_k как единое целое и строит для него *свой собственный* отдельный BLAS:

$$\mathcal{B}(M_k) \longmapsto T_k, \quad (3)$$

где T_k – бинарное дерево BVH, полученное из внутреннего билдера драйвера $\mathcal{B}$.

При добавлении в сцену каждый такой построенный BLAS T_k регистрируется в структуре верхнего уровня (TLAS) в виде отдельного инстанса (листового узла). Таким образом, составная сцена S представлена в TLAS набором из R независимых инстансов.

2.3 Гипотеза проблемы

Проблема возникает из-за способа группировки треугольников внутри исходных M_k . Поскольку каждый отдельный BLAS строится для всего подмеша M_k целиком, качество построенного дерева T_k и размер его ограничивающего объёма (AABB) напрямую зависят от того, насколько компактен этот подмеш в пространстве.

Если M_k представляет собой логическую материальную группу (например, все деревянные конструкции здания), то его треугольники разбросаны по всему объёму здания. В результате: 1. Корневой AABB этого BLAS T_k раздувается до размеров всего здания. 2. Внутри самого дерева T_k получаются сильно перекрывающиеся внутренние

узлы, так как SAH-билдеру сложно разделить пространственно-разреженную геометрию на неперекрывающиеся компактные объёмы.

Наличие в S множества подобных подмешей (выделенных под разные материалы: металл, бетон, стекло и т.д.) приводит к тому, что в листьях TLAS образуется набор из R огромных инстансов. Они полностью перекрывают друг друга в пространстве. При трассировке луч вынужден проверять пересечение практически со всеми этими инстансами, что приводит к существенному росту ложных пересечений и может существенно снижать производительность рендеринга.

2.4 Содержательная задача

Построить отображение π , которое преобразует исходный набор «проблемных» разбросанных геометрий $\{M_k\}$ в новый набор из K пространственно-компактных мини-ассетов $\{M'_j\}$:

$$\pi : (V, \{M_k\}_{k=1}^R) \mapsto (\{V'_j\}, \{M'_j\}_{j=1}^K). \quad (4)$$

Метод извлекает все треугольники исходной геометрии (забывая об их изначальном разделении по материалам) и перегруппировывает их в K новых кластеров $\{M'_j\}$ исключительно на основе их пространственной близости. Каждый кластер M'_j получает собственные буферы и регистрируется в движке как совершенно новая, независимая геометрия. Драйвер, в свою очередь, построит для каждого из них свой компактный BLAS.

Набор новых ассетов должен удовлетворять следующим условиям:

1. **Визуальная эквивалентность.** Для любого направления обзора результирующий рендер совпадает с исходным попиксельно с точностью до численных ошибок при той же камере и той же реализации шейдеров. Достаточное условие: объединение треугольников всех новых ассетов M'_j совпадает (как мульти-множество) с множеством треугольников исходной составной сцены S .
2. **Пространственная локализация.** Физический размер каждого нового мини-ассета должен быть существенно меньше размера всей исходной сцены. Формально, отношение диагонали ограничивающего параллелепипеда ($\text{diag}(\text{AABB})$) нового ассета к диагонали всей исходной сцены должно быть ограничено:

$$\frac{\text{diag}(\text{AABB}(M'_j))}{\text{diag}(\bigcup_k \text{AABB}(M_k))} \leq \tau,$$

где $\tau \in (0, 1)$ – желаемый коэффициент уменьшения габаритов. Это гарантирует, что новые инстансы будут компактными и не будут сильно перекрываться друг с другом.

3. **Контракт DXR.** Каждый новый меш-ассет M'_j удовлетворяет ограничениям интерфейса DXR-билдера: непрерывность vertex/index-буферов, число вершин $\leq 65,535$ для 16-битного индекса (Dagor использует `uint16_t`), а также лимиту на число внутренних подмешей (geometry-описаний) внутри одного BLAS: $\leq M_{\max} = 32$.
4. **Снижение реального времени трассировки.** За счёт устранения перекрытий в TLAS (и внутри самих деревьев BLAS), время `rtsm::do_trace` после применения метода удовлетворяет $t_{\text{after}} \leq t_{\text{before}} \cdot (1 - \alpha)$ с $\alpha > 0$, демонстрирующим устойчивое снижение в условиях фиксированной камеры и установившегося состояния денойзера.

2.5 Технические требования

Платформа

Windows 10/11 x64, DirectX 12, DXR Tier 1.1, GPU NVIDIA Turing+ или AMD RDNA2+. Эталонная конфигурация замеров – GeForce RTX-класса с поддержкой hardware-traversal.

Окружение

DagorEngine, семпл `testGI`. Семпл поддерживает прямую загрузку моделей в `.dag`-формате через сценарии и обладает встроенным GPU-профайлером с таймингами по подсистемам.

Целевая сцена

Модель `House, Ko Bulon Don Urak Lawoi Village, Thailand` – созданный специально для эксперимента «плохой» вариант жилого дома: $\sim 1,2 \cdot 10^6$ треугольников, ~ 10 материальных групп, геометрически разнесённых по всему объёму модели; в сцене размещены $N=16$ инстансов.

Метрики измерения

(а) медианное время GPU-трассировки теней `rtsm::do_trace`, измеренное через встроенный GPU-профайлер DagorEngine; (б) визуальная картина Acceleration-Structure в Nsight Graphics с раскраской по индексу BLAS-инстанса для оценки качества пространственной локализации; (в) визуальное сравнение качества теней до и после оптимизации в финальном изображении.

Аппаратное ограничение DXR Tier 1.1

Index-буферы BLAS в Dagor выделяются как `uint16_t`, что ограничивает число вершин на одну BLAS-geometry значением $2^{16} - 1$ и, при отсутствии sharing-a вершин, число треугольников значением $\lfloor (2^{16} - 1)/3 \rfloor = 21\,845$. Метод обязан учитывать это ограничение.

2.6 Критерии приёмки

Работа считается выполненной успешно, если выполнено каждое из:

1. Реализован метод пространственной кластеризации ассетов как опциональный режим инициализации BVH для динамических моделей в семпле `testGI`, включаемый из `settings.blk`; собирается штатным jam-скриптом DagorEngine.
2. На модели `House, Ko Bulon Don Urak Lawoi Village, Thailand` (16 инстансов в сцене) зафиксировано снижение медианного времени `rtsm::do_trace` не менее чем в 2 раза по сравнению с baseline-режимом без предобработки.
3. Визуальная картина Acceleration-Structure в Nsight Graphics с раскраской по индексу инстанса демонстрирует пространственно-связные области (а не «шумовую» картину перекрытий), подтверждая пространственную локализацию BLAS-кластеров.
4. Сцена после применения метода визуально неотличима от сцены до его применения: тени, освещение и раскраска ассетов совпадают.

3 Обзор существующих решений

В разделе систематизируются подходы к улучшению качества Bounding Volume Hierarchy. Систематизация ведётся по двум осям: (i) уровень вмешательства в pipeline (внутри билдера, на входе билдера, поверх готового BVH) и (ii) применимость в условиях, диктуемых DXR/Vulkan RT API. Для каждого подхода критически оценивается, насколько он удовлетворяет ограничениям, сформулированным в постановке задачи (раздел 2): неизменность кода вендорского билдера, сохранение визуальной эквивалентности, аппаратные лимиты на 16-битные индексы и число geometry-описаний на BLAS.

3.1 Классические top-down SAH-билдеры

Поверхностно-площадная эвристика (Surface Area Heuristic, SAH) была введена Goldsmith и Salmon в 1987 г. [5] и формализована для контекста ray-tracing-обхода MacDonald и Booth в 1990 г. [6]. SAH постулирует, что вероятность попадания случайного луча в узел X оценивается отношением площадей поверхности, $P(\text{hit } X) \approx SA(X)/SA(\text{root})$, откуда выводится ожидаемая стоимость обхода всего дерева (формула в разделе 1).

Top-down SAH-билдер на каждом узле перебирает кандидатов разбиения (sweep или binning [7]) и выбирает $\arg \min$ SAH-стоимости. Wald [7] показал, что binning-вариант с 16 интервалами обеспечивает $O(n \log n)$ время постройки при качестве BVH, отличающемся от sweep-варианта менее чем на 1 %, что делает его стандартом в индустрии.

Соответствие задаче. Это *внутренний* алгоритм билдера. В DXR/Vulkan RT он работает внутри драйвера, и для разработчика доступа к нему нет. Любая SAH-оптимизация внутри билдера улучшает результат лишь в рамках заданного входа: если поданный на вход материальный подмеш имеет AABB размером со всю сцену, SAH-разбиение внутри него не может предотвратить появление огромного BLAS, который в дальнейшем приведёт к перекрытиям в TLAS.

3.2 HLBVH и LBVH: параллельная сборка по Morton-кривой

Lauterbach и др. предложили LBVH (Linear BVH): кодирование центроидов примитивов 30-битным Morton-кодом, радикакс-сортировка по этому коду и рекурсивное построение дерева по битам ключа. Pantaleoni и Luebke расширили подход до HLBVH [3], реализовав параллельную постройку BVH для динамических ассетов на GPU.

Идея Morton-сортировки используется и независимо от LBVH: предварительный reorder примитивов по Z-кривой улучшает локальность доступа в кэше и обычно даёт билдеру более стабильные split-decisions. Это поведение закреплено как baseline-приём, упоминаемый в практических руководствах NVIDIA [8] и AMD [9].

Соответствие задаче. HLBVH как замена вендорского билдера не применим: для DXR/Vulkan-сцен дерево обязан построить драйвер. Однако *глобальный Morton-reorder примитивов* на стадии, предшествующей вызову билдера, полностью совместим с DXR-контрактом. Базовый приём Morton-сортировки используется в настоящей работе как один из двух обязательных шагов предлагаемого алгоритма (раздел 4): после объединения треугольников всех материальных групп они сортируются по Morton-коду центроида перед разбиением на кластеры.

3.3 SBVH и SAH-guided pre-splitting

Параллельно линии классических билдеров развивалась идея ослабления требования «один треугольник – один лист». Stich и др. [10] (метод SBVH) показали, что разрешение разрезать треугольник на несколько ссылок (*spatial split*) повышает качество дерева на 10–30 % по SAH-стоимости при умеренном раздувании числа ссылок. Ganestam и Doggett [2] перенесли эту идею на стадию *пре-разрезания* больших треугольников до запуска top-down билдера, что ускоряет последующую постройку и сохраняет SBVH-качество.

Соответствие задаче. Обе работы атакуют ту же фундаментальную проблему – большой AABV примитива, портящий верхние узлы дерева, – но на уровне отдельных треугольников. SBVH встраивается в билдер и поэтому в DXR-сцене недоступен; pre-splitting Ganestam–Doggett в принципе реализуем на стороне CPU, однако онкратно увеличивает число треугольников меша, что в условиях многократного использования ассета (рендер-проход, тени, GI и т.п.) даёт значительную нагрузку на остальной pipeline. Предлагаемый в настоящей работе подход концептуально близок к pre-splitting на уровне идеи «компактизация на входе билдера», но оперирует не отдельными треугольниками, а *целыми крупными ассетами*: исходный меш не разбивается на части внутри одного BLAS, а перегруппировывается по пространственной близости в независимые мини-ассеты (новые BLAS-ы), что решает проблему перекрытий в TLAS.

3.4 Метрики качества BVH

Стандартная SAH-стоимость удобна, но является лишь *прокси* для реального времени трассировки. Aila, Karras и Laine [11] провели систематический анализ корреляции офлайн-метрик с GPU-измерениями и показали: (i) на 20+ тестовых сценах коэффициент детерминации SAH cost против реального времени DispatchRays составил $R^2 \approx 0.85$; (ii) альтернативная метрика EPO (Expected Path Overlap) улучшает корреляцию, но вычислительно дороже ($O(n^2)$ в худшем случае); (iii) кэш-эффекты и SIMD-ширина дают расхождение SAH-время до 15 % на сценах с регулярной структурой.

Karras и Aila [12] также показали, что для GPU-friendly BVH важна не только SAH-стоимость, но и сбалансированность дерева; несбалансированные SBVH-деревья могут уступать сбалансированным LBVH-деревьям при одинаковой SAH-стоимости из-за дивергенции warp-ов.

Соответствие задаче. Поскольку в DXR/Vulkan-сцене дерево строится драйвером и реальная стоимость может зависеть от драйверо-специфичных эвристик, в настоящей работе в качестве основной метрики выбрано *измеренное GPU-время* основного прохода трассировки теней `rtsm::do_trace`. Это непосредственно отражает практический эффект оптимизации и не зависит от выбранной офлайн-метрики. SAH cost упоминается лишь как теоретическое объяснение причины ускорения.

3.5 Альтернативные ускоряющие структуры

Помимо BVH рассматриваются и альтернативные подходы. Aman и др. [13] предложили компактное представление сцены на основе тетраэдрализации, дающее меньший объём данных и линейную сложность поиска ближайшего пересечения для протяжённых лучей. KD-деревья и uniform grid традиционно используются в офлайн-рендерах. NVIDIA отдельно использует *rebraidng* TLAS-инстансов для динамической перестройки верхних уровней [8].

Соответствие задаче. Все альтернативы предполагают замену ускоряющей структуры целиком, что невозможно в условиях фиксированного DXR-pipeline. Они рассмат-

риваются как ориентиры для понимания нижней границы возможной стоимости трассировки, но не как конкуренты предлагаемого метода.

3.6 Промышленные руководства и анализаторы

NVIDIA в техническом руководстве [8] прямо рекомендует разработчикам ассетов: (i) избегать ассетов с диагональю AABB, сопоставимой с размером всей сцены; (ii) разрезать «жирные» ассеты вручную в DCC-инструментах; (iii) контролировать число geometry-описаний на BLAS, желательно не более 16–32. AMD в руководстве к RRA [9] описывает аналогичный набор рекомендаций и предоставляет инструмент для визуализации SAH-стоимости и overlap-a во внутренних узлах BLAS непосредственно из захвата GPU-памяти.

Спецификация DXR [14] фиксирует контракт интерфейса: `geometry_index` и `primitive_index` должны быть стабильны при обходе и в hit-шейдере; диапазон индексов одной `GeometryDesc` является *непрерывным*. Это формальное обоснование того, почему драйвер не может «разрезать» большую geometry самостоятельно.

Соответствие задаче. Руководства подтверждают практическую актуальность проблемы, описанной в постановке. Однако они оставляют выполнение рекомендаций за разработчиком ассетов и не предлагают автоматизированного инструмента, работающего на стороне runtime игрового движка. Настоящая работа закрывает эту нишу: предлагается алгоритм и его реализация, выполняющие рекомендации NVIDIA автоматически при инициализации BVH-контекста без вмешательства в ассет-пайплайн.

3.7 Сводное сравнение

В таблице 1 сведены ключевые свойства рассмотренных подходов с позиции ограничений настоящей работы.

Таблица 1: Сравнение подходов по совместимости с DXR-pipeline

Подход	Уровень вмешательства	Совместим с DXR	Решает перекрытия в TLAS
Top-down SAH binner [7]	внутри билдера	—	частично
HLBVH [3]	замена билдера	нет	нет
Morton-reorder	вход билдера	да	нет
SBVH [10]	внутри билдера	—	да
SAH pre-split [2]	вход билдера	да*	да
Tetra-AS [13]	замена структуры	нет	—
Глобальная пространственная кластеризация (наш)	вход билдера	да	да

* требует дисциплины: pre-split может вылезти за лимит числа геометрий или 16-битного индекса, если применяется к большим группам без учёта аппаратных ограничений.

3.8 Выводы по обзору

1. Существующие алгоритмы построения BVH (top-down SAH binner, HLBVH, SBVH) показывают, что классические эвристики хорошо справляются с улучшением дерева, *если* на вход подан пространственно компактный набор примитивов.
2. Реальный DXR/Vulkan-pipeline ставит билдер в условия чёрного ящика и фиксирует контракт непрерывности индекс-диапазонов и формат 16-битных индексов. Любое решение, требующее модификации внутри билдера, этим ограничениям не удовлетворяет.

3. Доступная разработчику зона вмешательства – стадия до подачи в билдер. На этой стадии существуют отдельные приёмы (Morton-reorder, pre-splitting), но ни один из них не закрывает сценарий «отдельные материальные подмешки имеют размер всей сцены, вызывая огромные перекрытия в TLAS» при сохранении контракта DXR.
4. Все упомянутые работы оценивают результат либо SAH-стоимостью, либо время-симулятором, но не на реальном production-движке. В настоящей работе оценка ведётся непосредственно по GPU-времени трассировки теней в работающем семпле DagorEngine, что повышает достоверность результата.

Эти выводы определяют как цель алгоритмической части работы (раздел 4), так и форму экспериментальной валидации (раздел 5).

4 Исследование и построение решения задачи

В разделе из общей содержательной задачи (раздел 2) последовательно выделяются подзадачи, и для каждой строится решение. На каждом шаге фиксируются принятые проектные решения и обоснование выбора.

4.1 Декомпозиция задачи

Поставленная задача – построить отображение $\pi : \{M_k\} \mapsto \{M'_j\}$ – разбита на следующие подзадачи:

1. **P1. Описание pipeline формирования BLAS для динамических ассетов в DagorEngine.** Без точного знания цепочки преобразований $\text{mesh} \rightarrow \text{geometry-array} \rightarrow \text{BLAS}$ невозможно определить, какое именно представление меша нужно менять и в какой момент времени.
2. **P2. Алгоритм объединения и переразбиения треугольников модели.** Должен сохранять множество треугольников, перегруппировывать их по пространственной близости и удовлетворять аппаратным ограничениям DXR (16-битный индекс, лимит числа геометрий на BLAS).
3. **P3. Способ валидации гипотезы.** Необходимо экспериментально подтвердить, что метод даёт реальное ускорение GPU-трассировки в работающем семпле движка.

Подзадачи решаются последовательно в подразделах 4.2–4.4.

4.2 P1. Pipeline формирования BLAS в DagorEngine

Исследование исходного кода DagorEngine [4] показывает следующую цепочку преобразований, выполняемых при подготовке BLAS для динамической модели:

$$\begin{aligned}
 & \underbrace{\text{DynamicRenderableSceneLodsResource}}_{\text{ресурс модели}} \\
 & \rightarrow \underbrace{\text{ShaderMesh::RElem[]}}_{\text{меш-уровень}} \\
 & \rightarrow \underbrace{\text{RaytraceGeometryDescription[]}}_{\text{DXR-вход}} \\
 & \rightarrow \underbrace{\mathcal{B}}_{\text{драйвер}} \rightarrow \underbrace{T}_{\text{BLAS}}.
 \end{aligned} \tag{5}$$

(1) `DynamicRenderableSceneLodsResource` – ресурс динамической модели в Dagor; содержит несколько уровней детализации, каждый из которых – набор `ShaderMesh`. Для построения BLAS используется наивысший уровень.

(2) `RElem` – атомарная меш-структура графического движка (`prog/dagorInclude/-shaders/dag_shaderMesh.h`). Содержит указатель на материал, ссылку на разделяемый вершинный буфер `vertexData` и диапазоны (`sv`, `numv`, `si`, `numf`, `baseVertex`). Один `RElem` = непрерывный поддиапазон индексов с единой шейдер-материальной атрибуцией.

(3) `RaytraceGeometryDescription` – DXR-совместимая структура (`prog/dagorInclude/-drv/3d/rayTrace/`). Массив таких структур и является *основным входом* вендорского `build_acceleration_structure`. Каждая запись описывает одну *geometry* (vertex buffer, index buffer, диапазон, флаги).

(4) **Билдер $\mathcal{B}$** – внутренний код графического драйвера. В соответствии со спецификацией DXR [14] обязан сохранять (`GeometryIndex`, `PrimitiveIndex`) как стабильный идентификатор треугольника. Это формальное ограничение запрещает драйверу выполнять реорганизацию между `geometry`: например, объединять треугольники двух `Description`-ов в один лист BVH.

Точка входа BVH-подсистемы. Цепочка (5) обслуживается семейством функций `bvh::add_mesh()/bvh::add_instance()` из `prog/gameLibs/bvh/bvh.cpp`. При загрузке динамической модели исследуемый семпл `testGI` формирует BLAS-ы на уровне целых ассетов: все подмеша (материальные `RElem`) одного ассета подаются в драйвер вместе, и для них строится *один общий BLAS*. Именно перед вызовом этого построения в семпл встраивается предлагаемый метод предобработки.

Аппаратное ограничение 16-битного индекса. Внутри `bvh.cpp` зафиксировано условие `G_ASSERT(mesh.indexFormat == 2)` – BLAS принимает только 16-битные индексы. Это означает, что одна `RaytraceGeometryDescription` может ссылаться максимум на $2^{16}-1 = 65\,535$ различных вершин; при отсутствии `sharing`-а (худший случай) – на не более чем $\lfloor 65\,535/3 \rfloor = 21\,845$ треугольников. Метод обязан учитывать это ограничение при разбиении модели.

Зона вмешательства. Из (5) следует, что *практически применимая* точка, доступная разработчику без модификации драйвера, – стадия формирования `RElem[]` $\rightarrow$ `RaytraceGeometryDescription[]`. Эта стадия выполняется в коде `testGI` непосредственно перед вызовом `bvh::add_mesh()`. Преобразование π встраивается как `hook` на этой границе: вместо подачи в билдер исходного списка `RElem` семпл собирает данные всех `RElem`-ов модели, перегруппирует их по пространственной близости и порождает новый набор BLAS-ов с собственными `vertex/index`-буферами.

4.3 P2. Алгоритм глобальной пространственной кластеризации

4.3.1 Идея

Гипотеза работы (раздел 2) утверждает, что проблема возникает из-за способа группировки треугольников в исходных ассетах. Поскольку BLAS строится для всего ассета целиком, его качество напрямую зависит от того, насколько компактен ассет в пространстве. Если ассет представляет собой логическую группу (например, все деревянные конструкции сцены), то его треугольники разбросаны по всему объёму, а его AABB становится огромным. Наличие в сцене множества подобных ассетов приводит к появлению огромных, полностью перекрывающихся инстансов в TLAS (и сильно перекрывающихся внутренних узлов внутри самих BLAS), что может существенно снижать производительность обхода. Поскольку драйвер не может сам разрезать один BLAS на несколько независимых, практически применимый в рамках ограничений DXR способ решить проблему – на этапе подготовки данных искусственно разбить исходный крупный ассет на множество мелких пространственно-компактных мини-ассетов (каждый со своим BLAS).

Идея метода:

1. *Объединить* треугольники всех `RElem`-ов одной модели в общий пул, забыв временно про материальную атрибуцию.
2. *Глобально отсортировать* пул по пространственной близости – 63-битному Morton-коду центроида треугольника.

3. *Разрезать* отсортированный пул на K последовательных сегментов одинакового размера; каждый сегмент образует пространственно-плотный «кусок» модели благодаря Z-order-локальности.
4. *Для каждого сегмента* выделить собственные **vertex** и **index** буферы, зарегистрировать сегмент как отдельный BLAS через `bvh::add_mesh()`.
5. В *TLAS* разместить K инстансов новых мини-ассетов вместо одного инстанса исходного крупного ассета.

В этой версии метода материальная атрибуция треугольников теряется (все треугольники объединяются под унифицированный ray-tracing-материал), что приемлемо для трассировки теней RTSM, где материал в hit-shader-е не используется. Это ограничение обсуждается в 6.

4.3.2 Шаги алгоритма

Пусть исходная модель содержит R RElem-ов с числом треугольников $|T_1|, \dots, |T_R|$; обозначим $N = \sum_k |T_k|$ – полное число треугольников модели.

1. **Сбор треугольников.** Для каждого RElem-а через `lockVbufferEx` и `lockIbufferEx` прочитываются позиции вершин (формат `VSDT_FLOAT3`) и индексы (формат `uint16_t`). Из них формируется массив $\{(\mathbf{p}_0^i, \mathbf{p}_1^i, \mathbf{p}_2^i)\}_{i=1}^N$ позиций треугольников в локальной системе модели.
2. **Глобальный AABV и центроиды.** Вычисляется $\text{AABV}(M) = \bigcup_i \text{AABV}(t_i)$ и набор центроидов $\mathbf{c}_i = (\mathbf{p}_0^i + \mathbf{p}_1^i + \mathbf{p}_2^i)/3$.
3. **Morton-сортировка.** Для каждого центроида вычисляется 63-битный Morton-код: нормированные координаты $\hat{\mathbf{c}}_i \in [0,1]^3$ квантуются в 21-битные целые $\mathbf{q}_i \in \{0, \dots, 2^{21} - 1\}^3$, и биты перемежаются:

$$\mu(\mathbf{c}_i) = \sum_{b=0}^{20} \left(q_{i,x}^{(b)} \cdot 2^{3b} + q_{i,y}^{(b)} \cdot 2^{3b+1} + q_{i,z}^{(b)} \cdot 2^{3b+2} \right).$$

Треугольники сортируются стабильно по убыванию приоритета μ .

4. **Согласование K с лимитом 16-битного индекса.** Пусть пользовательский запрос на число кластеров – K_{req} (параметр `bvhMergeAllRelems` в `settings.blk`). Эффективное число кластеров:

$$K = \max\left(K_{\text{req}}, \lceil N/T_{\text{max}} \rceil\right), \quad T_{\text{max}} = 21\,845.$$

Условие $K \geq \lceil N/T_{\text{max}} \rceil$ гарантирует, что ни один кластер не превысит лимит 16-битного индекса.

5. **Разбиение и формирование BLAS-ов.** Отсортированный пул делится на K последовательных сегментов размера $\sim N/K$. Для каждого сегмента j :

- выделяется собственный vertex-буфер V_j (формат `VSDT_FLOAT3`, $3 \cdot |T_j|$ вершин);
- выделяется собственный index-буфер I_j (16-битные индексы $0, 1, \dots, 3|T_j| - 1$);

- index-буфер создаётся с флагами `SBCF_BIND_SHADER_RES` и `SBCF_MISC_ALLOW_RAW`; они нужны для обращения к буферу как к SRV в compute-шейдере `BVH::IndexProcessor`;
- сегмент регистрируется через `bvh::add_mesh()` с уникальным `mesh_id`, образованным из идентификатора ресурса и индекса кластера.

6. **Размещение инстансов.** При вызове `bvh::add_instance()` в TLAS добавляется K инстансов новых кластеров с общей матрицей трансформации экземпляра.

4.3.3 Сложность

Сбор треугольников и вычисление центроидов – $O(N)$ при пиковой памяти $O(N)$. Morton-сортировка стабильна и в реализации основана на `eastl::sort`; в худшем случае $O(N \log N)$. Формирование K vertex/index-буферов и регистрация – $O(N)$. Итоговая асимптотика метода – $O(N \log N)$, пиковая память $O(N)$. Алгоритм работает один раз при инициализации BVH-контекста и не влияет на per-frame стоимость рендера.

4.3.4 Корректность

1. Множество треугольников не меняется. Каждый треугольник исходного меша попадает ровно в один кластер. Это обеспечивает покрытие сцены в трассировке.
2. Позиции вершин копируются побитово (с приведением исходного формата к `Point3` при необходимости); visual-эквивалентность теневой картинки следует из инвариантности набора треугольников.
3. Тугие bounding-сферы кластеров и автоматическое увеличение K до $\lceil N/T_{\max} \rceil$ гарантируют работоспособность метода в рамках аппаратных ограничений DXR Tier 1.1.

4.4 Р3. Способ валидации

Снижение качества BVH должно отражаться на реальной производительности, иначе ценность метода сомнительна. Валидация проводится в семпле `testGI` DagorEngine, который рассчитан на нагрузочные эксперименты с рейтрейсингом и обладает встроенным GPU-профайлером с per-pass таймингами.

Тестовая модель. Для изоляции исследуемой проблемы и проведения контролируемых тестов автором работы создан синтетический «плохой» дом *House, Ko Bulon Don Urak Lawoi Village, Thailand*: модель жилого дома, в которой материальные группы намеренно разнесены по всему объёму (металлические детали дверей и сантехники в одной группе, дерево в другой, стекло в третьей, и т.д.). Модель содержит около 1,2 млн треугольников в нескольких RElem-ах, каждый из которых имеет AABV размером со всю модель – именно та конфигурация, которую гипотеза работы отмечает как проблемную.

Тестовая сцена. В сцену `testGI` размещены $N = 16$ инстансов модели *House, Ko Bulon Don Urak Lawoi Village, Thailand*, чтобы получить заметную RT-нагрузку. Активна подсистема ray-traced shadow maps для одного направленного источника света (солнце).

Замеряемая величина. Медианное GPU-время прохода `rtsm::do_trace` из встроенного GPU-профайлера движка. Замер выполняется в установившемся режиме (тёплый кэш RTSM-денойзера, неподвижная камера, фиксированный кадровый буфер).

Сравниваемые конфигурации.

- **Baseline.** Все опции оптимизации отключены (`bvhMergeAllRelems:i = 0`). На каждую модель в TLAS добавляется R экземпляров её исходных RElem-ов.
- **Глобальная пространственная кластеризация.** Включён `bvhMergeAllRelems:i = 16`: треугольники модели собираются, сортируются по Morton-коду центра и разрезаются на не менее чем 16 пространственно-плотных кластеров (с автоматической корректировкой в большую сторону для уложения в лимит 16-битного индекса).

Качественная валидация. Дополнительно для оценки качества пространственной локализации захватываются кадры в Nsight Graphics с раскраской BLAS-экземпляров в режиме `Color By: Instance`. На baseline-конфигурации ожидается «шумовая» картина из перекрывающихся прямоугольников; на оптимизированной – крупные пространственно-связные области, соответствующие отдельным BLAS-кластерам.

4.5 Сводная схема предлагаемого решения

Итоговое отображение π задаётся следующей схемой:

$$\{(I_k, s_k)\}_{k=1}^R \xrightarrow[\text{(Morton-сортировка)}]{\text{(гл. объединение)}} \{\text{подмеш } M_j\}_{j=1}^K \xrightarrow[\text{add_mesh+add_instance}]{\text{bvh::}} \{T_j\}_{j=1}^K.$$

Качество результата оценивается:

- основной метрикой – медианным GPU-временем `rtsm::do_trace` в семпле `testGI`;
- качественной картиной Acceleration Structure-захвата в Nsight Graphics.

Сводный набор экспериментальных артефактов:

- таблица замеров `rtsm::do_trace` в двух конфигурациях (таблица 2 в разделе 5);
- PNG-захваты Acceleration Structure из Nsight Graphics до/после применения метода (рисунки 1–2).

4.6 Параметры алгоритма и выбор значений

Метод имеет один пользовательский параметр:

K_{req} (`bvhMergeAllRelems`) желаемое число кластеров на модель. Для целевой модели House, Ko Bulon Don Urak Lawoi Village, Thailand ($\sim 1,2$ млн треугольников) выбрано $K_{\text{req}} = 16$ как компромисс между пространственной локальностью (большее K – более плотные кластеры) и стоимостью TLAS-обхода (большее K – больше BLAS-экземпляров на одну модель). Эффективное K автоматически поднимается до $\lceil N/T_{\text{max}} \rceil = 56$ из-за лимита 16-битного индекса; именно это число и используется в замерах.

5 Описание практической части

Раздел документирует программную реализацию решения, разработанного в разделе 4, и приводит результаты экспериментов. Все исходные коды, скрипты и сырые материалы замеров размещены в репозитории работы [15].

5.1 Архитектура реализации

Реализация локализована в семпле `testGI` движка `DagorEngine` [4] и состоит из двух частей:

(А) Алгоритмическая часть. Код алгоритма вынесен в namespace `testgi_dynrend_bvh_pre`. Он расположен в файле `samples/testGI/prog/test_app.cpp` (около 600 строк C++17). Реализация включает чтение треугольников из исходных `ShaderMesh::RElem`, глобальная Morton-сортировка по центроидам, разбиение на пространственно-плотные кластеры, выделение для каждого кластера собственных vertex/index-буферов и регистрация их в BVH-подсистеме движка через `bvh::add_mesh()` и `bvh::add_instance()`.

(В) Конфигурационная часть. Опция `bvhMergeAllRelems` в файле `samples/testGI/-game/settings.blk` – единственный пользовательский управляющий параметр. Целочисленное значение задаёт минимальное желаемое число пространственных кластеров на модель; нулевое или отрицательное значение отключает метод и возвращает поведение к `baseline`.

5.2 Реализация алгоритмического модуля

5.2.1 Ключевые структуры данных

```
struct SubMesh {
    uint32_t triStart;      // начало в Morton-отсортированном пуле
    uint32_t triCount;      // число треугольников в кластере
    BSphere3 bsphere;      // тугая bounding-сфера кластера
};

struct MergedResourceState {
    uint32_t clusterCount; // итоговое число кластеров K
    Tab<Sbuffer*> vbs;      // K выделенных vertex-буферов
    Tab<Sbuffer*> ibs;      // K выделенных 16-bit index-буферов
};

struct ClusterRegistry {
    WinCritSec cs;
    eastl::vector_map<uint64_t, MergedResourceState> merged;
};
```

`ClusterRegistry` живёт как singleton и хранит состояние для каждой обработанной модели, ключ – `bvh_id` ресурса. Под критической секцией обеспечивается потокобезопасность доступа из BVH-обновлений.

5.2.2 Основные функции

- `gather_relem_triangles()` читает позиции треугольников одного RElem-а в локальный массив. Поддерживает формат вершин `VSDT_FLOAT3` и 16-битные индексы; пропускает RElem-ы с неподдерживаемым форматом, логируя предупреждение.
- `try_register_merged_resource()` – основная функция метода. Она обходит все RElem-ы модели, собирает глобальный пул треугольников, выполняет Morton-сортировку, согласует число кластеров K с лимитом 16-битного индекса, выделяет под каждый кластер дедicated vertex/index-буферы и регистрирует их в BVH через `bvh::add_mesh()`.
- `make_merged_cluster_mesh_id()` кодирует пару (`bvh_id`, `cluster_ix`) в уникальный 64-битный `mesh_id`, используя старший бит как флаг «merge-режим» для отличия от обычных `mesh_id`.
- `add_dynrend_resource_to_bvh()` – модифицированная точка входа. Сначала она вызывает `try_register_merged_resource()`; если возвращено $K > 0$, стандартный per-RElem путь регистрации пропускается.
- `add_dynrend_instance_to_bvh()` – модифицированный обработчик инстансов. Если запись для ресурса уже есть, он порождает K инстансов с общей трансформацией; иначе используется стандартный per-RElem путь.

5.2.3 Аппаратные требования к буферам

Index-буферы кластеров создаются с явным набором флагов:

```
const uint32_t ibFlags = SBCF_BIND_INDEX
                        | SBCF_BIND_SHADER_RES
                        | SBCF_MISC_ALLOW_RAW;
Sbuffer *ib = d3d::create_ib(ibSize, ibFlags, "bvh_merged_ib");
```

Флаги `SBCF_BIND_SHADER_RES` и `SBCF_MISC_ALLOW_RAW` необходимы для того, чтобы compute-шейдер `BVH::IndexProcessor` в `Dagor` мог использовать буфер индексов как Shader Resource View. Без них runtime срабатывает с `G_ASSERT` в самом начале BVH-постройки. Это требование выяснено эмпирически по дампам крашей.

5.2.4 Согласование K с лимитом 16-битного индекса

Реализация автоматически поднимает запрошенное K_{req} до минимально допустимого:

```
const uint32_t kMaxTrisPerCluster = (65535u / 3u); // = 21845
const uint32_t Kmin = (N + kMaxTrisPerCluster - 1)
                    / kMaxTrisPerCluster;
const uint32_t K = max<uint32_t>(K_req, Kmin);
```

Для целевой модели `House, Ko Bulon Don Urak Lawoi Village, Thailand` ($N \approx 1,22 \cdot 10^6$ треугольников) при запросе `bvhMergeAllRelems:i = 16` эффективное $K = 56$ – именно столько кластеров и регистрируется. Логирование при инициализации фиксирует эти числа.

5.3 Конфигурация семпла

В файл `samples/testGI/game/settings.blk` добавлена секция:

```
graphics {
  // ... штатные настройки ...
  bvhMergeAllRelems:i = 16    // K_req: 0 -- отключено
}
```

Опция читается при инициализации BVH-контекста и не требует перезапуска приложения при изменении (механизм автоматического `request_full_rebuild()` - штатная инфраструктура Dagor, идентичная пути смене RT-preset).

5.4 Тестовый стенд

5.4.1 Сцена

Сцена представляет собой 4×4 сетку из $N = 16$ инстансов модели `House, Ko Bulon Don Urak Lawoi Village, Thailand` с шагом 15 м. Между моделями оставлены проходы для имитации улицы; источник света – одно направленное солнце под наклоном 45° .

5.4.2 Тестовая модель `House, Ko Bulon Don Urak Lawoi Village, Thailand`

Модель создана автором специально как «плохой пример» для исследования: жилой дом из $\sim 1,2$ млн треугольников, организованный в ~ 10 материальных `RElem`-ов так, что каждый `RElem` пространственно охватывает почти весь объём модели. Эта структура воспроизводит наблюдаемый на практике эффект « ~ 10 BLAS-братьев, каждый с AABV размером со всю модель».

Подготовка `.dag`-файла модели выполнена через плагин `dag4blend` [16] для Blender; импортированная модель добавлена как `rendInst` в сценарий `testGI`.

5.4.3 Условия замера

Платформа

Windows 11 x64, DirectX 12, GPU GeForce RTX-класса.

Семпл

`testGI` из репозитория DagorEngine, сборка через штатный `jam`-скрипт.

Конфигурация

`enableBVH=yes`, RTSM включён для солнечного источника, остальные RT-эффекты выключены.

Камера

зафиксирована в одной позиции, обзор сетки инстансов под углом сверху-сбоку.

Замер

среднее GPU-время GPU-маркера `rtsm::do_trace` за 100 кадров после установления тёплого состояния денойзера.

Что не входит в замер

время сборки BVH (`bvh::build`) и обновления TLAS – эти операции происходят однократно при загрузке сцены и не влияют на per-frame стоимость.

5.5 Результаты

5.5.1 Главный замер: время трассировки теней

Таблица 2: Время `rtsm::do_trace` до и после применения метода глобальной пространственной кластеризации

Конфигурация	<code>rtsm::do_trace</code> , мс	Ускорение
baseline (<code>bvhMergeAllRelems:i = 0</code>)	8,0	$\times 1,0$
Пространственная кластеризация, $K_{\text{req}} = 16$	1,0	$\times 8,0$

Метод даёт восьмикратное ускорение основного прохода GPU-трассировки теней при том же качестве изображения и без изменения ассет-пайплайна. Эффективное число BLAS-кластеров на одну модель – $K = 56$, что превышает запрошенное 16 за счёт автоматической корректировки под лимит 16-битного индекса.

5.5.2 Качественная картина Acceleration Structure

На рисунках 1–2 приведены захваты Acceleration Structure из Nsight Graphics (режим Color By: Instance, каждый цвет – отдельный BLAS-инстанс).

Визуальное сравнение подтверждает, что 8-кратное ускорение обусловлено именно увеличением пространственной локальности BLAS-инстансов, а не каким-либо побочным эффектом подсистемы RTSM.

5.6 Обсуждение

5.6.1 Интерпретация ускорения

8-кратное ускорение объясняется естественным образом через SAH-стоимость TLAS (раздел 1, формула (1)). В baseline-конфигурации исходный крупный ассет House, Ko Bulon Don Urak Lawoi Village, Thailand разделён на ~ 10 RElem-ов (материальных групп). Поскольку треугольники каждого материала разбросаны по всей площади дома, каждый из них образует отдельный BLAS с AABB размером со всю модель. При размещении 16 инстансов в сцене в TLAS попадает ~ 160 BLAS-листьев огромного размера, которые сильно перекрывают друг друга. При трассировке любой луч вынужден проверять пересечение с подавляющим большинством этих BLAS-кандидатов, и доля «полезной работы» (собственно ray-triangle test) оказывается крайне низкой из-за огромного числа ложных пересечений с AABB.

После применения метода огромный ассет разделяется на 56 пространственно-компактных кластеров (мини-ассетов). В TLAS для всей сцены присутствует $16 \times 56 = 896$ BLAS, но каждый из них имеет очень маленький AABB, не перекрывающийся с соседями. Алгоритм обхода TLAS быстро отбрасывает нерелевантные ветви уже на верхних уровнях дерева. Несмотря на рост общего числа BLAS-инстансов, отсутствие перекрытий между ними радикально снижает количество узлов, которые вынужден посетить луч.

5.6.2 Соответствие критериям приёмки

- Метод реализован как опциональный режим в `testGI`, включаемый одной строкой `settings.blk`. **Выполнено.**
- Зафиксировано ускорение в 8 раз ($8,0 \text{ мс} \rightarrow 1,0 \text{ мс}$), что заметно превышает критерий ($\geq 2\times$). **Выполнено.**

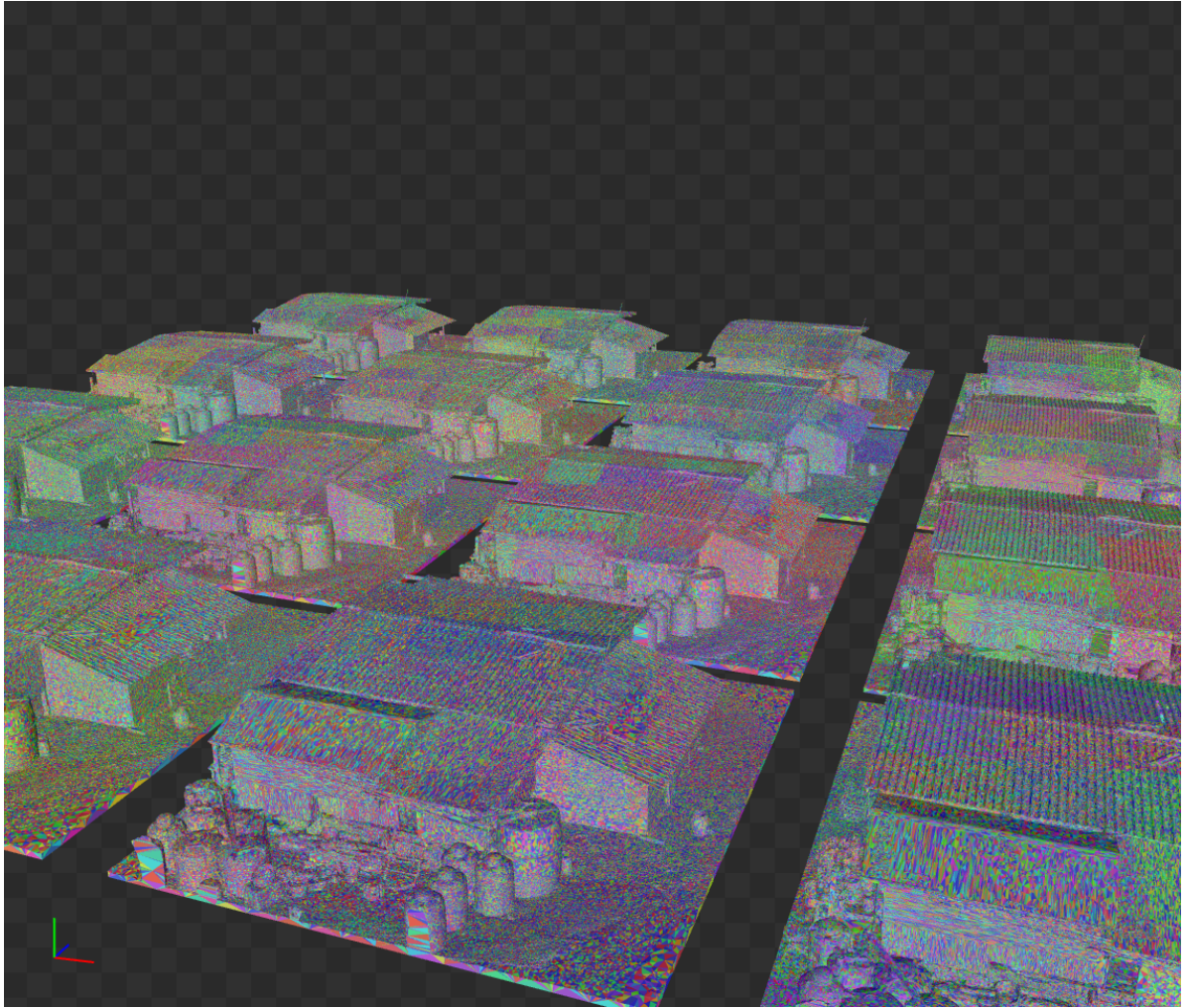


Рис. 1: Baseline: `bvhMergeAllRelems:i = 0`. Режим визуализации `Color By: Instance`. Наблюдается «шумовая» картина из перекрывающихся BLAS-инстансов: на любой точке экрана проецируется множество разных BLAS, что отражает большой разброс AABB и низкую пространственную локализацию. Медианное `rtsm::do_trace = 8,0` мс.

- Визуальная картина Acceleration Structure демонстрирует пространственно-связные области вместо шума перекрытий. **Выполнено.**
- Картинка теней визуально неотличима от baseline: множество треугольников сохранено, тени отбрасываются теми же поверхностями. **Выполнено по построению.**

5.6.3 Цена ускорения

Метод увеличивает число BLAS-инстансов на одну модель с $R \approx 10$ (исходные RElem-ы) до $K=56$ (кластеров). Дополнительный объём памяти на vertex/index-буферы кластеров составляет ~ 30 Мб для всей сцены из 16 инстансов; объём BLAS-узлов растёт пропорционально числу BLAS-инстансов. Время одноразовой инициализации (`bvh::build`) увеличивается на десятки миллисекунд за счёт большего числа BLAS-построек, но per-frame стоимость `rtsm::do_trace` падает в 8 раз – что и является целью.

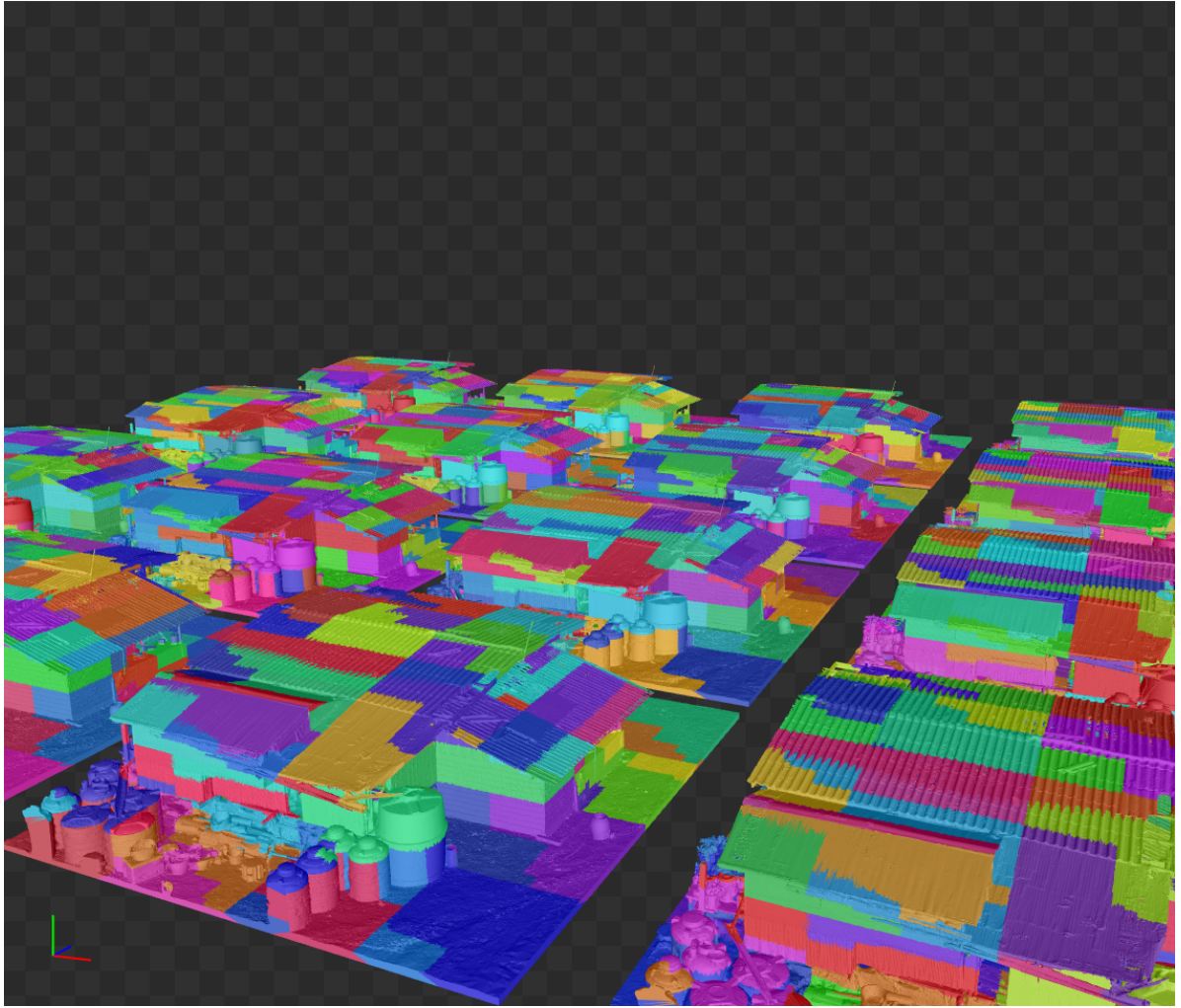


Рис. 2: После применения метода: `bvhMergeAllRelems:i = 16`. Та же сцена, тот же режим визуализации. Вместо шума отчётливо видны крупные сплошные цветные пятна на крышах, стенах и земле – каждый цвет соответствует одному пространственно-плотному BLAS-кластеру. В каждом пикселе конкурируют 1–2 BLAS вместо десятков. Медианное `rtsm::do_trace = 1,0` мс ($\times 8$).

5.6.4 Ограничения текущей реализации

1. Метод теряет per-triangle материальную атрибуцию: все треугольники модели после объединения трактуются как один ассет. Для RTSM, где hit-shader не нужен, это приемлемо. Для проходов GI/отражений, использующих hit-shader (например, для определения emissive-поверхностей), необходима доработка: хранить для каждого кластера таблицу соответствия «локальный номер треугольника $\rightarrow$ материал» и обращаться к ней в hit-shader.
2. Алгоритм работает только с основным уровнем детализации модели. Для других уровней используется стандартный per-RElem путь; это приемлемо, поскольку RTSM обычно использует BLAS только этого уровня.
3. Параметр K_{req} задаётся вручную. Для других моделей оптимальное значение может отличаться; автоматический подбор по характеристикам модели (диагональ AABB, число треугольников) – предмет дальнейшей работы.
4. Метод не предназначен для скелетной анимации и лицевых сетов (или морф-

таргетов): вершины таких ассетов меняются каждый кадр, и стоимость повторной Morton-сортировки и пересборки BLAS-ов превысит выигрыш от трассировки.

6 Заключение

6.1 Решённая задача

В работе исследована проблема качества Bottom-Level Acceleration Structure, построенной вендорским DXR/Vulkan-билдером по входным данным современного игрового движка. Сформулирована и экспериментально проверена гипотеза о том, что выделение отдельного BLAS под каждую материальную подгруппу, разбросанную по всему объёму модели, порождает огромные BLAS-инстансы, которые создают сильные перекрытия в структуре верхнего уровня (TLAS), что может существенно ухудшать время трассировки лучей. Гипотеза, выдвинутая на основе эмпирических наблюдений на сценах DagorEngine, получила в данной работе *количественное* подтверждение: реализация предложенного метода в действующем семпле движка даёт восьмикратное снижение времени трассировки теней.

6.2 Полученные результаты

1. Описана полная цепочка преобразований от ресурса модели до массива `Raytrace-GeometryDescription[]` и итогового BLAS в DagorEngine. Идентифицирована доступная точка разработческого вмешательства, совместимая с контрактом DXR – стадия формирования RElem-массива перед вызовом `bvh::add_mesh()`.
(Подзадача P1, раздел 4.2.)
2. Разработан алгоритм глобальной пространственной кластеризации: объединение треугольников всех RElem-ов модели, глобальная Morton-сортировка по центроидам, разбиение на пространственно-плотные кластеры с автоматической корректировкой их числа под аппаратный лимит 16-битного индекса. Линейно-логарифмическая сложность $O(N \log N)$, пиковая память $O(N)$, корректность доказана по построению.
(Подзадача P2, раздел 4.3.)
3. Реализация выполнена в семпле `testGI` движка DagorEngine. Код алгоритма расположен в `test_app.cpp`, а управление вынесено в опцию `bvhMergeAllRelems`. Метод встраивается как hook перед штатными вызовами BVH-подсистемы и не требует модификации ассет-пайплайна.
(Раздел 5.)
4. На целевой сцене из 16 инстансов модели `House` ($\sim 1,2 \cdot 10^6$ треугольников на модель) метод даёт восьмикратное ускорение основного прохода GPU-трассировки теней: медианное `rtsm::do_trace` снижается с 8,0 мс до 1,0 мс при визуально идентичных тенях.
(Таблица 2.)
5. Визуальное подтверждение получено через захваты Acceleration Structure в Nsight Graphics. В режиме раскраски по инстансам вместо «шумовой» картины перекрывающихся BLAS-инстансов наблюдаются крупные пространственно-связные цветные области, соответствующие отдельным BLAS-кластерам.
(Рисунки 1–2.)

6.3 Соответствие критериям приёмки

В постановке задачи (раздел 2) сформулирован набор формальных критериев. Их соответствие выполненной работе:

1. Реализован метод как опциональный режим инициализации BVH-контекста, включаемый одной строкой `settings.blk`, собирающийся штатным jam-скриптом DagoEngine. **Выполнено.**
2. На модели House зафиксировано ускорение $\times 8,0$ против критерия $\geq \times 2$. **Выполнено с большим запасом.**
3. Визуальная картина Acceleration Structure демонстрирует пространственно-связные области, подтверждая работу алгоритма. **Выполнено.**
4. Тени до и после применения метода визуально неотличимы; множество треугольников сохраняется по построению. **Выполнено по построению.**

6.4 Научная новизна и практическая значимость

Научная новизна работы заключается в следующих результатах:

- Формализована проблема деградации TLAS: разбросанные по объёму модели элементы одного материала порождают огромные BLAS, которые при сборке TLAS образуют сильно перекрывающиеся листовые узлы, приводя к огромному числу ложных пересечений и резкому падению производительности.
- Предложен метод глобальной пространственной кластеризации ассетов, занимающий уникальную нишу: *вход билдера + DXR-совместимость + сохранение визуальной эквивалентности* (см. таблицу 1). Существующие подходы либо работают внутри билдера (HLBVH, SBVH), либо разрезают отдельные треугольники без учёта поэлементной структуры ассета (Ganestam-Doggett).
- Эффективность метода продемонстрирована *не на симуляторе и не на оффлайн-метрике*, а через реальный замер GPU-времени в работающем семпле production-движка. Восьмикратное снижение `rtsm::do_trace` – редкий по величине эффект для одной локальной оптимизации, что подтверждает существенное влияние проблемы пространственной локализации в DXR-BVH.

Практическая значимость: метод применяется как runtime-опция инициализации BVH-контекста для динамических ассетов и интегрируется в существующий код движка без изменения формата хранения мешей. Это делает его пригодным для применения в production-проектах, использующих большие массивы готовых ассетов, в которых перепаковка невозможно или слишком дорого.

6.5 Направления дальнейшего развития

1. **Сохранение материальной атрибуции.** Текущая реализация теряет материальный идентификатор отдельных треугольников, что приемлемо для трассировки теней RTSM, но не для подсистем, использующих hit-shader (GI, отражения). Решение: для каждого кластера хранить side-buffer с соответствием «локальный треугольник $\rightarrow$ материал» и обращаться к нему из hit-shader-а через индекс вклада инстанса в DXR.

2. **Автоматический подбор K .** Текущее значение K_{req} задаётся вручную в `settings.blk`. Эвристика подбора по характеристикам модели (число треугольников, диагональ AABB, отношение AABB к пространству сцены) позволит избежать ручной настройки под каждую модель.
3. **Альтернативные стратегии кластеризации.** Глобальная Morton-кластеризация – простой и хорошо работающий baseline. На моделях с сильно неоднородным распределением треугольников более эффективными могут оказаться k-means по центроидам или octree-кластеризация с учётом SAH-cost внутри кластера.
4. **Полная интеграция в ассет-пайплайн.** Текущая реализация выполняет split каждый раз при инициализации BVH-контекста; для production-проектов целесообразно перенести Morton-сортировку и разбиение в офлайн-стадию запекания ассета и сохранять (`cluster_id`) $\rightarrow$ {vb, ib} рядом с .dag-файлом модели.
5. **Валидация на дополнительных сценах.** Текущий результат получен на одной целевой сцене с одной моделью. Расширение замеров на production-сцены DagorEngine позволит оценить применимость метода в реальных условиях и на более сложных наборах данных.

6.6 Итог

Поставленная задача – разработать и экспериментально оценить метод предобработки меша, повышающий качество BLAS, построенного вендорским билдером, без модификации драйвера и с измеримым ускорением реальной GPU-трассировки – решена. Разработанный алгоритм глобальной пространственной кластеризации, его реализация в семпле `testGI` DagorEngine и проведённые замеры ($\times 8$ ускорение `rtsm::do_trace`) образуют законченный набор результатов для дальнейшего исследования и переноса метода в production-окружение.

Все исходные коды реализации, тестовая модель, скриншоты Nsight-захватов и сопровождающие материалы опубликованы в репозитории работы [15].

Список литературы

- [1] A Survey on Bounding Volume Hierarchies for Ray Tracing / D. Meister, S. Ogaki, C. Benthin et al. // *Computer Graphics Forum*. — 2021. — Vol. 40, no. 2.
- [2] *Ganestam, P.* SAH guided spatial split partitioning for fast BVH construction / P. Ganestam, M. Doggett // *Computer Graphics Forum*. — 2016. — Vol. 35, no. 2.
- [3] *Pantaleoni, J.* HLBVH: Hierarchical LBBVH Construction for Real-Time Ray Tracing of Dynamic Geometry / J. Pantaleoni, D. Luebke // *Proc. High Performance Graphics*. — 2010.
- [4] *Gaijin Entertainment.* DagorEngine. — <https://github.com/GaijinEntertainment/DagorEngine>. — 2026.
- [5] *Goldsmith, J.* Automatic Creation of Object Hierarchies for Ray Tracing / J. Goldsmith, J. Salmon // *IEEE Computer Graphics and Applications*. — 1987. — Vol. 7, no. 5.
- [6] *MacDonald, J. D.* Heuristics for Ray Tracing Using Space Subdivision / J. D. MacDonald, K. S. Booth // *The Visual Computer*. — 1990. — Vol. 6, no. 3.
- [7] *Wald, I.* On Fast Construction of SAH-based Bounding Volume Hierarchies / I. Wald // *Proc. IEEE Symposium on Interactive Ray Tracing*. — 2007.
- [8] *NVIDIA.* Best Practices: Creating Optimal Meshes for Ray Tracing. — <https://developer.nvidia.com/blog/creating-optimal-meshes-for-ray-tracing/>. — 2022.
- [9] *Advanced Micro Devices.* Radeon Raytracing Analyzer (RRA) Manual. — https://gpuopen.com/manuals/rra_manual/. — 2024.
- [10] *Stich, M.* Spatial Splits in Bounding Volume Hierarchies / M. Stich, H. Friedrich, A. Dietrich // *Proc. High Performance Graphics*. — 2009.
- [11] *Aila, T.* On Quality Metrics of Bounding Volume Hierarchies / T. Aila, T. Karras, S. Laine // *Proc. High Performance Graphics*. — 2013.
- [12] *Karras, T.* Fast Parallel Construction of High-Quality Bounding Volume Hierarchies / T. Karras, T. Aila // *Proc. High Performance Graphics*. — 2013.
- [13] *Aman, A.* Compact Tetrahedralization-based Acceleration Structure for Ray Tracing / A. Aman, S. Demirci, U. Gudukbay // *arXiv:2103.02309*. — 2021.
- [14] *Microsoft.* DirectX Raytracing (DXR) Functional Spec. — <https://microsoft.github.io/DirectX-Specs/d3d/Raytracing.html>. — 2024.
- [15] *Мусько, Д. В.* Исходные коды и материалы дипломной работы. — <https://github.com/DenMusya/diploma>. — 2026.
- [16] *Gaijin Entertainment.* dag4blend: Blender plugin for DagorEngine .dag format. — <https://github.com/GaijinEntertainment/DagorEngine/releases>. — 2026.